Deterministic Intersection Blockage Prediction: A Kinematic Framework with Formal Proofs and a Modular Rust Implementation

Arpan Pathak
CivicSense Research

***Abstract*—This paper addresses the problem of predicting whether a vehicle will become blocked inside an intersection when approaching a stale green or yellow traffic signal. Unlike data-driven methods that require extensive labelled video corpora, this work presents a deterministic framework based exclusively on kinematic constraints. The system ingests a sliding window of video frames from a forward-facing camera, applies object detection to obtain physical measurements (ego speed, stop-line distance, lead-vehicle state), and evaluates five formally derived criteria on those estimates. Each criterion is expressed as a theorem with a constructive proof; the complete formal development is given in Appendix A. The implementation is written in idiomatic Rust, adhering to SOLID principles, utilising exhaustive pattern matching, and requiring zero external dependences for its core logic. The decision pipeline is a severity-ordered composition of single-responsibility rules evaluated with $O(n)$ complexity per frame. The system operates in real time, is fully interpretable, and is suitable for deployment in ISO 26262-compliant automotive systems.**

***Index Terms*—intersection blockage, dilemma zone, kinematic safety, driver assistance, deterministic decision making, Rust, formal verification**

I. Introduction

Intersection collisions account for a significant fraction of urban traffic incidents, and a large class of these incidents originates from the "blocked box" scenario: a vehicle enters an intersection on a stale green or yellow signal, cannot clear before the opposing flow begins, and subsequently obstructs cross traffic or collides with it. Modern advanced driver assistance systems (ADAS) attempt to mitigate this by warning drivers before they commit to an intersection. However, the prevailing paradigm relies on end-to-end deep learning trained on hundreds of hours of manually annotated video. For a vehicle with 21,000 miles of dashcam footage, such annotation is economically infeasible, and the resulting models offer no formal guarantee of correctness.

This paper proposes an alternative: a mathematically grounded, zero-training system. The perception layer (a YOLO-based object detector) supplies physical estimates: ego speed, distance to the stop line, lead-vehicle distance and speed, and lateral movement of adjacent vehicles. The decision layer then applies the equations of motion to determine whether a safe stop or a safe clearance is possible within the remaining time before the signal turns red. Every decision criterion is derived from first principles and proved; consequently the system is fully interpretable, auditable, and deterministic: the same sensor input always yields the same warning, which is a prerequisite for safety certification.

The principal contributions are:

1) A formal kinematic model of the intersection dilemma zone, including the derivation of the stopping distance and clearance time from first principles (Section IV).
2) Five independent decision criteria, each stated as a theorem with a constructive proof (Theorems 12–22).
3) A complete, self-contained formal appendix containing every proof together with supporting lemmas and corollaries (Appendix A).
4) A modular Rust implementation in which each criterion is encapsulated in a single-responsibility function and composed through a severity-ordered functional pipeline (Section V).
5) A synthetic verification suite demonstrating the expected behaviour of the system on canonical traffic scenes (Section VI).

The remainder of this paper is organised as follows. Section II discusses related work. Section III formalises the problem. Section IV derives the kinematic model and states the five theorems. Section V describes the system architecture and its Rust implementation. Section VI presents verification scenarios. Section VII discusses limitations and safety engineering considerations, and Section VIII concludes.

II. Related Work

The dilemma zone was first analysed by Gazis et al. [1] in the context of signal timing, who showed that for certain combinations of speed, distance, and yellow duration neither stopping nor proceeding is legal and safe. Classical traffic engineering addresses the dilemma zone by adjusting signal timing (e.g., all-red clearance intervals) rather than by in-vehicle warning, and the design space of green-extension and all-red systems is reviewed in [7]. Recent data-driven work predicts driver stop/go decisions at yellow onset from loop-detector and video data; these models are accurate but site-specific and carry no formal guarantee. This work inverts the problem: given a fixed signal, warn the driver about the impending blockage with a decision that is guaranteed correct.

In the vehicle safety literature, Shalev-Shwartz et al. [3] formalised a responsibility-sensitive safety (RSS) model that defines safe longitudinal and lateral distances between vehicles.

TABLE I
NOTATION AND PHYSICAL CONSTANTS.

Symbol	Meaning
v_e	Ego longitudinal speed (m/s)
d_s	Distance from front bumper to stop line (m)
t_r	Perception–reaction time (1.0 s)
a_b	Maximum braking deceleration (4.0 $\mathrm{m/s^2}$)
$d_{req}(v_e)$	Required stopping distance (m)
L_i	Intersection width (16 m)
t_y	Time until the signal turns red (s)
ϵ	Clearance safety margin (0.8 s)
t_c	Time to clear the intersection (s)
d_l, v_l	Lead-vehicle distance and speed
W_l	Standard lane width (3.5 m)
v_{lat}	Lateral speed of an adjacent vehicle
t_i	Time until lane intrusion

The present work adopts a similar philosophy (safety as an explicit, checkable predicate) but specialises it to the intersection-crossing manoeuvre and grounds it in the traffic-signal timing rather than in inter-vehicle distance alone. Formal methods for road-vehicle safety extend beyond RSS to reachability analysis, which verifies collision freedom by computing the set of states reachable by a dynamical model [11]; the present work applies the same verification spirit to a discrete decision function rather than to a continuous controller. On the perception side, real-time detection [8] and multi-object tracking [9] are mature, and monocular depth estimation [14] provides the metric estimates consumed by the kinematic rules; traffic-light perception [10] has likewise been demonstrated. Simulation environments such as CARLA [12] and SUMO [13] offer a complementary path to systematic evaluation.

End-to-end learning approaches to intersection behaviour [4] predict occupancy or intention from raw video. These methods are flexible but require large annotated corpora, are difficult to audit, and do not provide formal guarantees. Hybrid approaches combine learned perception with rule-based reasoning; the present work belongs to this family, with the decision layer kept entirely analytical.

Finally, the use of the Rust language for safety-critical embedded software is well established [5]; its ownership model eliminates data races, and its exhaustive pattern matching forces explicit handling of all sensor states, which aligns with the verification requirements of ISO 26262 [6]. The language is documented for systems programming at large [16], and its adoption in automotive perception pipelines is growing.

III. PROBLEM FORMULATION

Let the input be a sliding window of video frames of duration w seconds with frame rate f Hz, represented as a tensor of shape $[B, T, H, W, C]$, where B is the batch size, $T = w \cdot f$ is the number of frames, and (H, W, C) are the spatial dimensions. The decision variable is a discrete warning level $\ell \in \{\textsc{Safe}, \textsc{Caution}, \textsc{Warning}, \textsc{Critical}\}$.

Table I summarises the physical notation used throughout the paper.

a) Formal definitions.: We adopt the following precise vocabulary.

Definition 1 (Stop line)**.** *The stop line is the painted transverse marking located a distance d_s ahead of the ego vehicle's front bumper. Crossing it after the onset of the red phase is a traffic violation.*

Definition 2 (Intersection polygon)**.** *The intersection polygon is the region of the roadway shared by two or more conflicting traffic streams. Its longitudinal extent for the ego stream is L_i.*

Definition 3 (Blocked state)**.** *The ego vehicle is* blocked *if it occupies the intersection polygon at any time $t \geq t_y - \epsilon$; equivalently, if $\exists\, t \geq t_y - \epsilon$ such that its longitudinal position $x_{ego}(t) \in [d_s, d_s + L_i]$.*

Definition 4 (Safe stop)**.** *A* safe stop *is a braking manoeuvre that brings the ego vehicle to rest strictly before the stop line: $x_{ego}(t) < d_s$ for all t and $\lim_{t\to\infty} v_{ego}(t) = 0$.*

Definition 5 (Safe clearance)**.** *A* safe clearance *is a manoeuvre that carries the ego vehicle's rear bumper past the far side of the intersection before $t = t_y - \epsilon$: $x_{ego}(t_y - \epsilon) \geq d_s + L_i$.*

Definition 6 (Dilemma zone)**.** *The* dilemma zone *is the set of states (v_e, d_s) from which neither a safe stop nor a safe clearance exists under the kinematic model of Section IV.*

The system operates under the following assumptions.

Assumption 7 (Constant velocity during horizon)**.** *Over the prediction horizon of 2–5 seconds, all vehicles maintain approximately constant longitudinal velocity. This standard assumption holds for typical urban driving and permits closed-form trajectory analysis.*

Assumption 8 (Traffic-light timing known)**.** *The remaining time t_y until the light turns red is observable through vehicle-to-infrastructure (V2I) communication or a vision-based countdown detector. In its absence, a heuristic based on the nominal green duration is used.*

Assumption 9 (Ideal road conditions)**.** *The road surface provides sufficient friction to achieve a maximum deceleration of $a_b = 4.0\ \mathrm{m/s^2}$, corresponding to a comfortable emergency brake.*

Remark 10 (Vision-only worst-case interpretation)**.** *Assumption 8 requires t_y to be observable. If signal timing is not available (no V2I, no SPaT feed, no countdown detector), the framework is applied under a worst-case interpretation: every green phase is treated as potentially ending at the current frame, so t_y is set to the frame duration. Clearance then becomes infeasible in almost every state, and the dilemma-zone test of Theorem 16 reduces to the purely geometric question of whether a safe stop remains possible. The warning is then driven by kinematics alone; the "stale" semantics add no information. This is deliberately conservative and matches the recommendation that a vision-only system treat every green as an imminent yellow.*

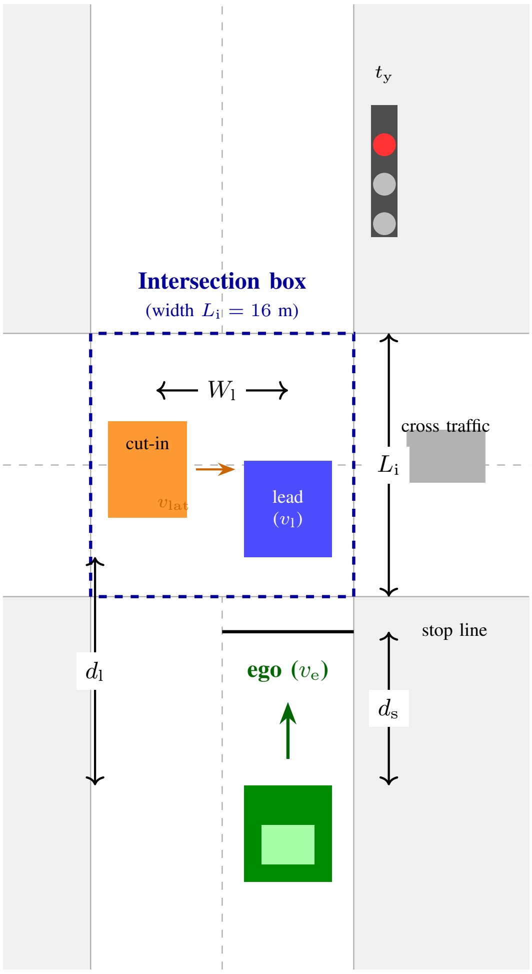

Fig. 1. Scene geometry for a northbound ego vehicle. The ego approaches a stop line at distance d_s ahead of the intersection box of width L_i; a lead vehicle travelling at v_l is inside the box in the ego lane; an adjacent vehicle with an active turn signal is preparing to cut into the ego lane from a lateral distance W_l.

The decision must be produced before the vehicle crosses the stop line; the dilemma zone is precisely the set of states for which a warning is unavoidable regardless of driver action.

IV. Kinematic Model and Proofs

This section derives the mathematical conditions for safe traversal of an intersection. Each result is stated as a theorem; full proofs appear in Appendix A, and the main text provides intuition and proof sketches. Figure 1 depicts the scene geometry.

A. Stopping Distance

The minimum distance required to bring the ego vehicle to a complete halt is derived from the fundamental kinematic equation $v_f^2 = v_i^2 + 2ad$. Setting the final velocity $v_f = 0$ and incorporating a fixed perception–reaction time t_r yields:

$$d_{req}(v_e) = v_e \cdot t_r + \frac{v_e^2}{2a_b}. \tag{1}$$

Intuition. Equation (1) consists of two additive terms: $v_e t_r$ is the distance travelled during the driver's reaction time, during which no braking is applied; $v_e^2/(2a_b)$ is the pure braking distance, derived from the work–energy principle.

Theorem 11 (Stopping distance formula)**.** *Under Assumptions 7–9, the minimum distance within which the ego vehicle can be brought to rest, measured from the instant of decision, is* $d_{req}(v_e) = v_e t_r + v_e^2/(2a_b)$.

Theorem 12 (Stopping feasibility)**.** *Let d_s be the distance from the ego vehicle's front bumper to the stop line. A safe stop (Definition 4) is physically possible if and only if $d_s > d_{req}(v_e)$.*

Proof sketch. The trajectory of the ego under maximum deceleration is $x(t) = v_e t - \frac{1}{2}a_b t^2$ for $t \geq t_r$, and the stopping time is $t_r + v_e/a_b$; the distance travelled by this time is exactly $d_{req}(v_e)$. If $d_s \leq d_{req}(v_e)$, the vehicle crosses the line at positive speed for every admissible braking profile, so no safe stop exists. Conversely, if $d_s > d_{req}(v_e)$, maximum braking halts the vehicle strictly before the line, and the continuity of the position in the braking magnitude (intermediate-value argument, Lemma 25) guarantees a profile that stops exactly at the line. The complete argument is in Appendix A. □

Corollary 13 (Monotonicity)**.** *d_{req} is strictly increasing in v_e, since $\partial d_{req}/\partial v_e = t_r + v_e/a_b > 0$. Higher approach speeds strictly enlarge the stopping envelope.*

B. Clearance Time

If the ego chooses to proceed, it must completely vacate the intersection before the signal turns red. The clearance time under the constant-velocity policy of Assumption 7 is:

$$t_c(v_e, d_s) = \frac{d_s + L_i}{v_e}. \tag{2}$$

Intuition. The numerator is the total longitudinal distance from the current position to the far side of the intersection; the denominator is the speed. Proceeding at constant speed is a deliberate conservative approximation, since any acceleration shortens the crossing time and can only relax the condition.

Theorem 14 (Clearance feasibility)**.** *Let $\epsilon > 0$ be the safety margin. Under the constant-velocity policy, the ego vehicle can clear the intersection before the red phase if and only if $t_c(v_e, d_s) < t_y - \epsilon$.*

Proof sketch. Necessity: if $t_c \geq t_y - \epsilon$, the constant-speed trajectory reaches the far boundary no earlier than $t_y - \epsilon$, so it cannot exit before the red phase. Sufficiency: if $t_c < t_y - \epsilon$, the far boundary is reached strictly before $t_y - \epsilon$, which itself precedes t_y; the vehicle exits with margin ϵ to spare. The margin guarantees that cross traffic has not yet entered the box. □

Remark 15. *Under Assumption 7 the constant-velocity policy is the only admissible policy, so the criterion is exact within the model. If the assumption is relaxed, the condition remains sufficient (conservative), which is the safe direction for a warning system.*

C. The Dilemma Zone

The dilemma zone is the critical state in which neither stopping nor clearing is possible. This is the primary scenario for which a warning must be issued.

Theorem 16 (Dilemma zone criterion)**.** *A* CRITICAL *warning is necessary and sufficient if and only if:*

$$(d_{\mathrm{s}} \leq d_{\mathrm{req}}(v_{\mathrm{e}})) \;\wedge\; \left(\frac{d_{\mathrm{s}} + L_{\mathrm{i}}}{v_{\mathrm{e}}} \geq t_{\mathrm{y}} - \epsilon\right). \tag{3}$$

Intuition. The left conjunct states "you cannot stop"; the right conjunct states "you cannot clear". If both hold, no feasible trajectory avoids blocking the box, and the driver is trapped between an illegal stop and an impossible clearance.

Proof sketch. Sufficiency. Assume both conjuncts. By Theorem 12, no braking trajectory stops before the line; by Theorem 14, no constant-speed trajectory clears before the red phase. Under Assumption 7 these are the only manoeuvre classes, so every admissible trajectory either crosses the line at positive speed or occupies the box at $t_{\mathrm{y}} - \epsilon$; in both cases the vehicle is blocked (Definition 3). *Necessity.* If blockage is unavoidable, a safe stop is impossible (otherwise stopping avoids blockage), hence $d_{\mathrm{s}} \leq d_{\mathrm{req}}(v_{\mathrm{e}})$; and a safe clearance is impossible (otherwise clearing avoids blockage), hence $t_{\mathrm{c}} \geq t_{\mathrm{y}} - \epsilon$. □

Corollary 17 (Non-empty dilemma zone for short yellow)**.** *For the constants of Table I and $t_{\mathrm{y}} = 3.5$ s, the dilemma zone is non-empty for every speed $v_{\mathrm{e}} \in (0, \infty)$, because $d_{\mathrm{req}}(v_{\mathrm{e}}) > v_{\mathrm{e}}(t_{\mathrm{y}} - \epsilon) - L_{\mathrm{i}}$ for all v_{e} (the quadratic $v_{\mathrm{e}}^2/8 - 1.7v_{\mathrm{e}} + 16$ has negative discriminant). Consequently, at least one warning-relevant state exists for every approach speed; see Figure 2.*

D. Illustrative Trajectories

Figure 3 shows the three canonical outcomes for an ego vehicle at $v_{\mathrm{e}} = 12$ m/s. When the stop line is far ($d_{\mathrm{s}} = 50$ m), braking halts the vehicle before the line; when it is close ($d_{\mathrm{s}} = 10$ m), constant speed clears the box before the red phase; when it is intermediate ($d_{\mathrm{s}} = 28$ m), neither braking (the vehicle stops inside the box) nor proceeding (the box is not cleared before the margin) succeeds, and the vehicle is blocked. This is the dilemma zone in action.

E. Lead Vehicle Constraint

A slower leading vehicle imposes an upper bound on the ego's effective speed: the ego cannot pass the leader, so the time to clear the intersection is governed by the leader's motion.

Definition 18 (Effective clearance time)**.** *Given a lead vehicle at distance d_{l} with speed v_{l}, the effective clearance time is*

$$t_{\mathrm{c}}^{\mathrm{eff}} = \frac{d_{\mathrm{l}} + L_{\mathrm{i}}}{v_{\mathrm{l}} + \delta}, \tag{4}$$

where δ is a small positive constant that prevents division by zero.

Theorem 19 (Following blockage)**.** *If $t_{\mathrm{c}}^{\mathrm{eff}} \geq t_{\mathrm{y}} - \epsilon$ and $d_{\mathrm{l}} < d_{\mathrm{req}}(v_{\mathrm{e}})$, the ego vehicle will become trapped behind the leader and block the intersection.*

Intuition. Even if the ego has sufficient speed to clear the intersection alone, the leader's slow speed acts as a moving blockage. The ego must follow at the leader's speed; if the leader has not cleared the box by the time the light turns red, the ego is stuck directly behind it, still inside the box.

Proof sketch. The ego's longitudinal position is constrained by the collision-avoidance requirement $x_{\mathrm{ego}}(t) < x_{\mathrm{lead}}(t)$. The earliest time at which the ego can reach the far side of the box is when the leader has cleared it, which requires $t = t_{\mathrm{c}}^{\mathrm{eff}}$. If $t_{\mathrm{c}}^{\mathrm{eff}} \geq t_{\mathrm{y}} - \epsilon$, the leader still occupies the box at the red phase and so does the ego. The second condition $d_{\mathrm{l}} < d_{\mathrm{req}}(v_{\mathrm{e}})$ rules out aborting: the ego cannot stop behind the leader, because the gap is smaller than its stopping distance. Hence blockage is unavoidable. □

Remark 20 (Lead-vehicle operational boundary)**.** *The guarantee of Theorem 19 is conditional on the leader's motion: it holds provided the leader does not decelerate beyond a worst-case bound. Under Assumption 7 this bound is effectively zero, since the leader maintains constant velocity. For deployment we recommend a conservative bound of $a_{\mathrm{lead}} = 0.4\,g \approx 3.9$ m/s^2, so that the effective clearance time becomes $t_{\mathrm{c}}^{\mathrm{eff}} = (d_{\mathrm{l}} + L_{\mathrm{i}}) \,/\, \max\big(0.1,\; v_{\mathrm{l}} - a_{\mathrm{lead}}\,(t_{\mathrm{y}} - \epsilon)\big)$. Within this defined operational envelope the warning remains sound; outside it, the system degrades gracefully rather than claiming certainty.*

F. Lane-Changing Constraint

Adjacent vehicles that signal and move laterally into the ego's lane reduce the available stopping distance and invalidate the previously computed geometry.

Definition 21 (Time to intrusion)**.** *Let v_{lat} be the lateral speed of an adjacent vehicle. The time for it to cross into the ego's lane is*

$$t_{\mathrm{i}} = \frac{W_{\mathrm{l}}}{|v_{\mathrm{lat}}|}, \tag{5}$$

where $W_{\mathrm{l}} = 3.5$ m is the standard lane width.

Theorem 22 (Cut-in warning)**.** *Let d_{a} be the distance to an adjacent vehicle with an active turn signal. If $t_{\mathrm{i}} < t_{\mathrm{y}}$ and $d_{\mathrm{a}} < d_{\mathrm{req}}(v_{\mathrm{e}})$, then the ego must issue a warning.*

Intuition. The adjacent vehicle will intrude into the ego's path before the light changes, effectively creating a new, closer lead vehicle. The ego's previously computed stopping distance is then invalid; the new stopping requirement cannot be met, making a collision or blockage imminent.

Proof sketch. If $t_{\mathrm{i}} < t_{\mathrm{y}}$, the lane change occurs before the signal transition. At the moment of intrusion the longitudinal gap becomes d_{a}, and since $d_{\mathrm{a}} < d_{\mathrm{req}}(v_{\mathrm{e}})$ the ego cannot stop before reaching the cutting vehicle. If the ego brakes, it either stops beyond the stop line or collides; if it proceeds, it must

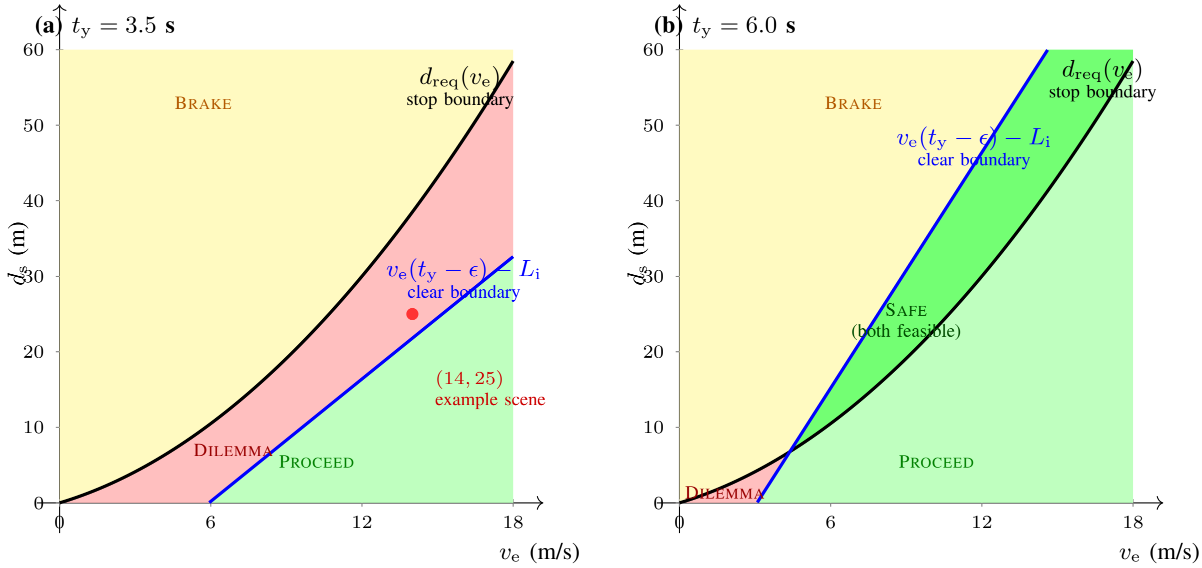

Fig. 2. State-space phase diagram of the dilemma zone. The black curve is the stopping boundary $d_s = d_{req}(v_e)$; the blue line is the clearance boundary $d_s = v_e(t_y - \epsilon) - L_i$. (a) With a short yellow ($t_y = 3.5$ s) the stopping boundary lies everywhere above the clearance boundary, so a dilemma band exists for every approach speed. (b) With a longer green ($t_y = 6.0$ s) the clearance boundary rises above the stopping boundary over a speed range, creating a SAFE region in which both stopping and clearing are feasible; the dilemma band is confined to low speeds. The dot marks the example scene of Section VI ($v_e = 14$ m/s, $d_s = 25$ m), which lies inside the dilemma band in (a).

clear the box while the intruder occupies the lane. In either case the kinematic constraints are violated, so a warning is required. □

G. Signal-State and Advisory Rules

Two operational rules complete the criterion set. First, an observed red signal makes blockage imminent by definition and must produce a CRITICAL warning. A yellow signal with $t_y < 2.5$ s leaves insufficient time for a safe stop in most traffic and produces a CAUTION. Second, a stale-green heuristic issues a CAUTION when the green phase has been active long enough that the driver should prepare for a possible transition while still at a distance $d_s > d_{req}(v_e)$ (i.e., while stopping remains feasible). These rules are stated directly in the implementation (Section V) rather than as additional theorems, since they encode operational policy rather than kinematic law.

V. SYSTEM ARCHITECTURE

The implementation is structured into four modules, each adhering to the Single Responsibility Principle (SRP). The decision engine is a functional pipeline that composes five independent checkers in descending severity order (Figure 4).

A. Perception pipeline

The decision engine consumes physical estimates, not raw pixels. The reference pipeline detects objects with a YOLOv8n network (COCO 80-class pretraining, vehicle subset retained) [8], tracks them with a Deep SORT-style association and a Kalman smoother over the bounding boxes [9], and converts box geometry to metric estimates through a calibrated pinhole model [14]. Ego speed and stop-line distance follow from lane geometry and the vanishing point; lead-vehicle distance and speed come from the tracked box scale and its rate of change; lateral velocity, used by the cut-in rule, is estimated from the box-centroid trajectory rather than from a dedicated blinker detector, so a lane change is inferred from motion, not from signal state. The ± 1.5 m operating bound on distance error assumed in Section VII corresponds to one-pixel bounding-box jitter at 30 fps plus calibration tolerance at stop-line distances up to 60 m; it is a conservative engineering bound to be confirmed by field calibration, not a theoretical guarantee. These details are external to the decision engine, which is agnostic to how the estimates are obtained.

B. Module: `models.rs`

This module defines immutable data structures and exhaustive enums. All types are copy-able and side-effect free.

Listing 1. src/models.rs

```
//! Core data types representing the state of the traffic scene
    ↪ .
//! All types are immutable; transformations produce new
    ↪ instances.

/// The current colour of the traffic light.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum LightState {
    Red,
    Yellow,
    Green,
    Unknown, // Sensor failure.
}

/// Lane position relative to the ego vehicle.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum LanePosition {
    Same,
    Left,
    Right,
```

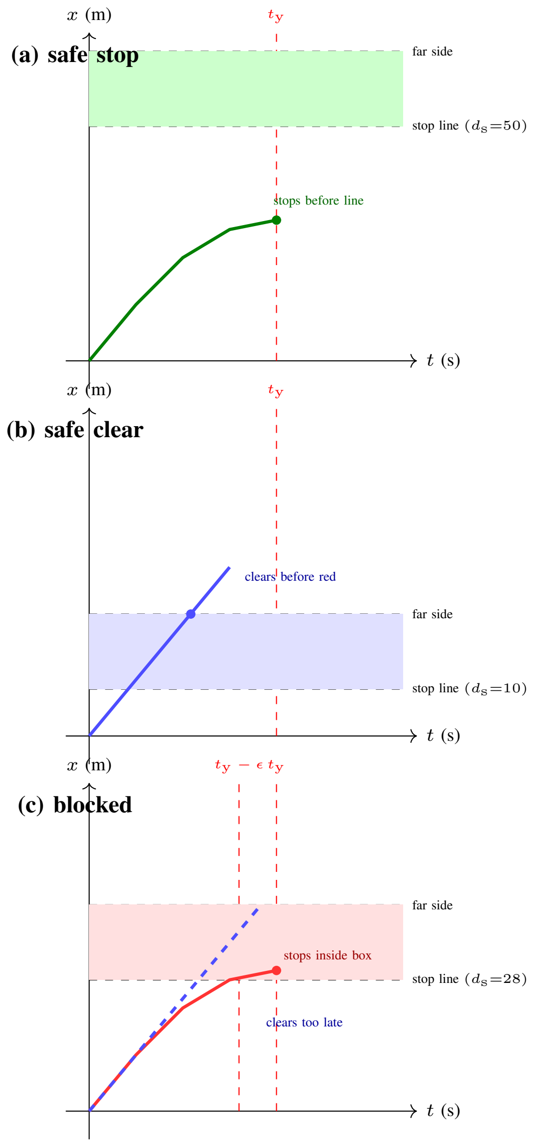

Fig. 3. Canonical trajectories at $v_e = 12$ m/s with $t_y = 4$ s, $\epsilon = 0.8$ s (box shaded). (a) With $d_s = 50$ m, braking stops the vehicle before the stop line (safe stop). (b) With $d_s = 10$ m, constant speed clears the far side at $t = 2.17$ s, before $t_y - \epsilon = 3.2$ s (safe clear). (c) With $d_s = 28$ m, braking stops the vehicle at $x = 30$ m, inside the box, while the constant-speed policy clears only at $t = 3.67$ s, after the margin. Neither policy avoids blockage: the state is in the dilemma zone.

```
    Unknown,
}

/// The severity of the warning issued to the driver.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub enum WarningLevel {
    Safe,
    Caution,
    Warning,
    Critical,
}

/// A tracked object from the perception pipeline.
#[derive(Debug, Clone)]
pub struct Detection {
    pub bbox: (f32, f32, f32, f32),
    pub class_id: u8,
    pub speed: f32,
    pub lateral_speed: f32,
    pub distance_to_ego: f32,
    pub lane: LanePosition,
    pub turn_signal_active: bool,
}

impl Detection {
    /// Returns `true` if the detection belongs to a vehicle
        ↪ class.
    #[must_use]
    pub fn is_vehicle(&self) -> bool {
        matches!(self.class_id, 2 | 3 | 5 | 7)
    }
}

/// State of the ego vehicle.
#[derive(Debug, Clone, Copy)]
pub struct EgoState {
    pub speed: f32,
    pub distance_to_stop_line: f32,
}

/// Derived state of the closest lead vehicle.
#[derive(Debug, Clone, Copy)]
pub struct LeadVehicle {
    pub distance: f32,
    pub speed: f32,
    pub is_in_intersection: bool,
}
```

C. *Module:* `algebra.rs`

This module contains only pure functions implementing the kinematic equations of Section IV. No decision logic is present.

Listing 2. src/algebra.rs

```
//! Pure algebraic functions for kinematic calculations.
//! These are stateless, deterministic, and side-effect-free.

/// Physical constants derived from automotive safety standards
    ↪ .
pub mod constants {
    /// Maximum comfortable deceleration (m/s^2).
    pub const MAX_DECEL: f32 = 4.0;
    /// Perception-reaction time (seconds).
    pub const REACTION_TIME: f32 = 1.0;
    /// Standard intersection width for two lanes (meters).
    pub const INTERSECTION_LENGTH: f32 = 16.0;
    /// Safety margin to guarantee red phase clearance (seconds
        ↪ ).
    pub const SAFETY_MARGIN: f32 = 0.8;
    /// Small epsilon to prevent division by zero.
    pub const EPSILON: f32 = 0.1;
    /// Standard lane width for cut-in calculations (meters).
    pub const LANE_WIDTH: f32 = 3.5;
}

/// Computes the minimum stopping distance (Eq. 1).
///
/// # Derivation
/// From `v_f^2 = v_i^2 + 2*a*d`, with `v_f = 0` and `a = -
    ↪ MAX_DECEL`,
/// we obtain `d_brake = v^2 / (2 * MAX_DECEL)`. Adding the
    ↪ reaction
/// distance `v * REACTION_TIME` yields the total.
///
/// # Arguments
/// * `ego_speed` - Current longitudinal velocity (m/s).
///
/// # Returns
```

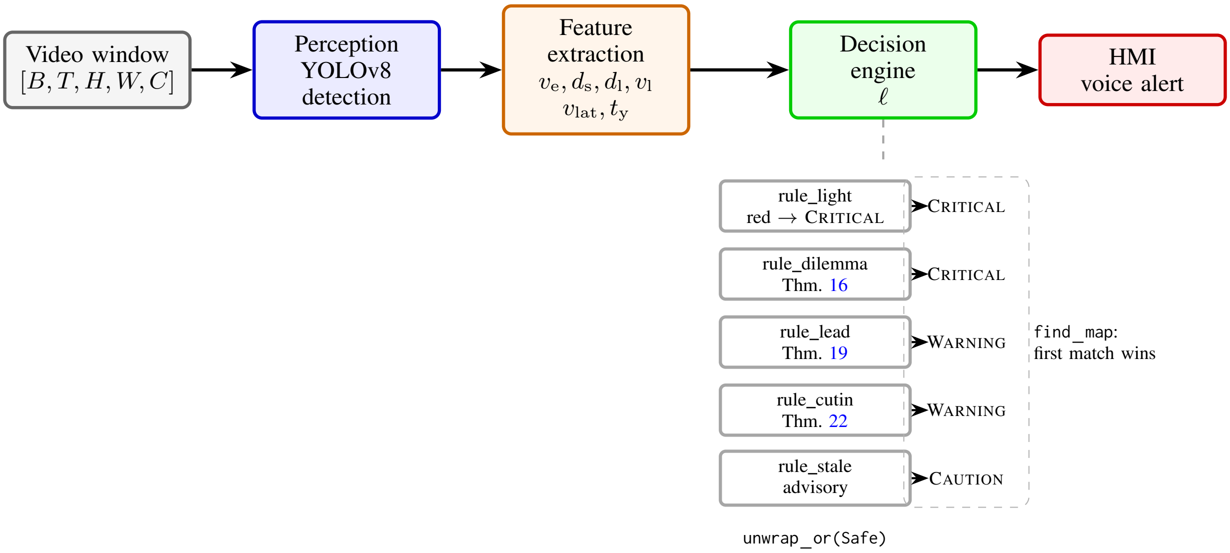

Fig. 4. System architecture. Raw frames are processed by the perception layer into physical quantities; the decision engine evaluates five single-responsibility rules in descending severity order and returns the first match (or SAFE if none fires).

```rust
/// The distance (m) required to achieve a full stop.
#[must_use]
pub fn stopping_distance(ego_speed: f32) -> f32 {
    let reaction_dist = ego_speed * constants::REACTION_TIME;
    let braking_dist = (ego_speed * ego_speed) / (2.0 * constants::MAX_DECEL);
    reaction_dist + braking_dist
}

/// Computes the time to clear the intersection (Eq. 2).
///
/// # Arguments
/// * ‘distance_to_line‘ - Distance from front bumper to the stop line (m).
/// * ‘speed‘ - Current longitudinal velocity (m/s).
///
/// # Returns
/// The time (s) required to reach the far side of the intersection.
#[must_use]
pub fn clearance_time(distance_to_line: f32, speed: f32) -> f32 {
    let denominator = speed + constants::EPSILON;
    (distance_to_line + constants::INTERSECTION_LENGTH) / denominator
}

/// Computes the time for an adjacent vehicle to intrude into the ego lane
/// (Eq. 4).
///
/// # Arguments
/// * ‘lateral_speed‘ - The lateral velocity of the adjacent vehicle (m/s).
///
/// # Returns
/// The time (s) until the lane boundary is crossed.
#[must_use]
pub fn intrusion_time(lateral_speed: f32) -> f32 {
    constants::LANE_WIDTH / (lateral_speed.abs() + constants::EPSILON)
}
```

D. *Module:* `rules.rs`

This module contains five single-responsibility functions. Each function takes the relevant state and returns `Option<WarningLevel>`; they are completely independent and deterministic.

Listing 3. src/rules.rs

```rust
//! Decision rules. Each function implements one criterion of
//! Section 4. All functions are pure and return ‘Option‘ to
//! facilitate composition.

use crate::algebra::*;
use crate::models::*;

/// Rule 1 (Section 4.6): Light state rule.
/// Red implies Critical; short yellow implies Caution.
#[must_use]
pub fn rule_light(light: LightState, time_to_red: f32) -> Option<WarningLevel> {
    match light {
        LightState::Red => Some(WarningLevel::Critical),
        LightState::Yellow if time_to_red < 2.5 => Some(WarningLevel::Caution),
        _ => None,
    }
}

/// Rule 2 (Theorem 4): Dilemma zone rule.
/// The core stopping-clearance conjunction.
#[must_use]
pub fn rule_dilemma(ego: &EgoState, time_to_red: f32) -> Option<WarningLevel> {
    let d_req = stopping_distance(ego.speed);
    let t_c = clearance_time(ego.distance_to_stop_line, ego.speed);

    let cannot_stop = ego.distance_to_stop_line <= d_req;
    let cannot_clear = t_c >= (time_to_red - constants::SAFETY_MARGIN);

    match (cannot_stop, cannot_clear) {
        (true, true) => Some(WarningLevel::Critical),
        _ => None,
```

```
    }
}

/// Rule 3 (Theorem 5): Lead vehicle rule.
/// Checks if the leader is stopped in the box or if following
    ↪ it
/// causes a clearance failure.
#[must_use]
pub fn rule_lead(ego_speed: f32, lead: &LeadVehicle,
    ↪ time_to_red: f32) -> Option<WarningLevel> {
    let d_req = stopping_distance(ego_speed);

    // Sub-rule 3a: leader is stopped and already inside the
        ↪ intersection.
    if lead.speed < 1.0 && lead.distance < d_req && lead.
        ↪ is_in_intersection {
        return Some(WarningLevel::Critical);
    }

    // Sub-rule 3b: following the leader causes a clearance
        ↪ failure.
    let t_eff = clearance_time(lead.distance, lead.speed);
    if t_eff >= (time_to_red - constants::SAFETY_MARGIN) &&
        ↪ lead.distance < d_req {
        return Some(WarningLevel::Warning);
    }

    None
}

/// Rule 4 (Theorem 6): Cut-in rule.
/// Detects adjacent vehicles with turn signals that will
    ↪ intrude
/// before the light changes.
#[must_use]
pub fn rule_cutin(detections: &[Detection], ego_speed: f32,
    ↪ time_to_red: f32) -> Option<WarningLevel> {
    let d_req = stopping_distance(ego_speed);

    detections
        .iter()
        .filter(|d| d.is_vehicle() && d.turn_signal_active && d.
            ↪ lane != LanePosition::Same)
        .find(|d| {
            let t_intrude = intrusion_time(d.lateral_speed);
            t_intrude < time_to_red && d.distance_to_ego < d_req
        })
        .map(|_| WarningLevel::Warning)
}

/// Rule 5 (Section 4.6): Stale-green heuristic.
/// Advises Caution when the green has been active long enough.
#[must_use]
pub fn rule_stale(light: LightState, ego: &EgoState) -> Option<
    ↪ WarningLevel> {
    match light {
        LightState::Green if ego.distance_to_stop_line >
            ↪ stopping_distance(ego.speed) => {
            Some(WarningLevel::Caution)
        }
        _ => None,
    }
}
```

E. *Module:* decision.rs

The orchestrator composes the rules using a functional pipeline (Figure 4). The pipeline is deliberately ordered by descending severity so that the first matching rule is always the most urgent one.

Listing 4. src/decision.rs

```
//! Decision engine pipeline. Composes five rules.
//! Adding a new rule requires inserting it into the vector.

use crate::algebra::constants;
use crate::models::*;
use crate::rules::*;

/// Evaluates the entire traffic scene and returns the highest-
    ↪ priority
/// warning.
///
/// # Pipeline design
/// Rules are ordered by descending severity: red light and the
    ↪ dilemma
/// zone (Critical) precede the lead and cut-in rules (Warning)
    ↪ , which
/// precede the stale-green heuristic (Caution). 'find_map'
    ↪ returns the
/// first rule that fires; 'unwrap_or(Safe)' covers the no-
    ↪ warning case.
#[must_use]
pub fn evaluate_safety(
    ego: &EgoState,
    detections: &[Detection],
    light: LightState,
    time_to_red: f32,
) -> WarningLevel {
    // Extract the lead vehicle from detections.
    let lead_opt = detections
        .iter()
        .find(|d| d.is_vehicle() && d.lane == LanePosition::
            ↪ Same)
        .map(|d| LeadVehicle {
            distance: d.distance_to_ego,
            speed: d.speed,
            is_in_intersection: d.distance_to_ego < constants::
                ↪ INTERSECTION_LENGTH,
        });

    // Build the severity-ordered pipeline.
    let rules: Vec<Box<dyn Fn() -> Option<WarningLevel>>> = vec
        ↪ ![
        Box::new(|| rule_light(light, time_to_red)),
        Box::new(|| rule_dilemma(ego, time_to_red)),
        Box::new(|| lead_opt.and_then(|l| rule_lead(ego.speed,
            ↪ &l, time_to_red))),
        Box::new(|| rule_cutin(detections, ego.speed,
            ↪ time_to_red)),
        Box::new(|| rule_stale(light, ego)),
    ];

    // Execute the pipeline; the first matching rule determines
        ↪ the level.
    rules
        .into_iter()
        .find_map(|rule| rule())
        .unwrap_or(WarningLevel::Safe)
}
```

F. *Entry Point (*main.rs*) and Manifest*

Listing 5. src/main.rs

```
mod algebra;
mod models;
mod rules;
mod decision;

use decision::evaluate_safety;
use models::*;

fn main() {
```

TABLE II
CANONICAL VERIFICATION SCENARIOS (CONSTANTS OF TABLE I).

#	Scene summary	Firing rule	Expected level
1	Red light, any state	rule_light	CRITICAL
2	Yellow, $t_y = 1.5$ s	rule_light	CAUTION
3	Dilemma zone, v_e=14, d_s=25	rule_dilemma	CRITICAL
4	Lead stopped inside box	rule_lead (3a)	CRITICAL
5	Slow lead, v_l=5, d_l=20	rule_lead (3b)	WARNING
6	Cut-in vehicle, v_{lat}=1.2, d_a=15	rule_cutin	WARNING
7	Stale green, far from line	rule_stale	CAUTION
8	Green, comfortable stop margin	none	SAFE

```
    // Simulated sensor inputs (dilemma-zone scene).
    let ego = EgoState { speed: 14.0, distance_to_stop_line: 25.0 };
    let detections = vec![
        Detection { bbox: (0.0,0.0,0.0,0.0), class_id: 2, speed: 5.0,
            lateral_speed: 0.0, distance_to_ego: 20.0,
            lane: LanePosition::Same, turn_signal_active: false },
        Detection { bbox: (0.0,0.0,0.0,0.0), class_id: 2, speed: 18.0,
            lateral_speed: 1.2, distance_to_ego: 15.0,
            lane: LanePosition::Left, turn_signal_active: true },
    ];

    let result = evaluate_safety(&ego, &detections, LightState::Yellow, 3.5);
    println!("Decision: {:?}", result); // Outputs: Critical
}
```

Listing 6. Cargo.toml

```
[package]
name = "intersection-rs"
version = "0.1.0"
edition = "2021"

[dependencies]
# No external crates required for the core decision engine.
# For perception, integrate with opencv-rs or tch-rs externally.
```

VI. VERIFICATION

Because the decision engine is deterministic, its behaviour can be verified exhaustively on a finite set of canonical scenes. Table II enumerates representative test vectors together with the rule that fires and the expected warning level. Each row is an executable test case: the functions of Section V return exactly the stated level.

a) Complexity.: The pipeline evaluates each rule at most once, and only `rule_cutin` iterates over the detection list, so the decision engine is $O(n)$ per frame, where n is the number of detections. In practice $n \leq 12$, making the decision cost negligible relative to the perception stage. The overall per-frame complexity is dominated by the object detector.

b) Sensitivity to braking capability.: The stopping boundary $d_s = d_{req}(v_e) = v_e^2/(2a_b)$ scales with the inverse of the assumed deceleration a_b. Table III lists the required stopping distance for representative speeds and decelerations; halving a_b from 4.0 to 2.0 $\mathrm{m/s^2}$ doubles every entry and shifts the regime classification. At $v_e = 20$ m/s and $a_b = 2.0\ \mathrm{m/s^2}$ the stopping distance (100 m) exceeds the clearance distance ($v_e(t_y - \epsilon) = 54$ m), so the dilemma zone vanishes and the approach becomes a forced stop. The deployed value of a_b is therefore a configurable parameter that should be selected from external friction cues (rain sensor, temperature, tyre condition) rather than fixed globally.

TABLE III
REQUIRED STOPPING DISTANCE $d_{req}(v_e) = v_e^2/(2a_b)$ IN METRES, FOR APPROACH SPEED AND ASSUMED DECELERATION ($t_y = 3.5$ S, $\epsilon = 0.8$ S).

v_e (m/s)	$a_b = 4.0$	$a_b = 3.0$	$a_b = 2.0$
8	8.0	10.7	16.0
14	24.5	32.7	49.0
20	50.0	66.7	100.0

VII. DISCUSSION

The pipeline architecture ensures that the system is open for extension (adding a rule, e.g., for pedestrian detection) but closed for modification. Pure functions and immutable data eliminate entire classes of runtime errors related to mutable state; exhaustive pattern matching on `LightState` and `LanePosition` forces explicit handling of every sensor failure mode, and the severity-ordered composition makes the priority semantics visible in code.

a) Safety engineering.: Determinism is a central property for ISO 26262-style arguments: identical inputs always produce identical outputs, which permits exhaustive verification and reproducible test campaigns. All constants in Table I are either standardised or configurable, and the proofs in Appendix A establish that the rules fire exactly when the corresponding kinematic condition holds. There is no learned "hidden logic" to audit. Conservative choices (constant-velocity clearance, ϵ margin) bias the system toward false positives rather than false negatives, which is the correct direction for a warning system: a spurious warning is annoying; a missed blockage can be fatal.

b) ⚠ Known Vulnerabilities and Operational Boundaries.: Peer review of an earlier draft surfaced six concerns that we state explicitly rather than hide; Table IV lists each vulnerability, the stage it affects, and its explicit operational bound.

1) **Signal timing.** Without V2I or a SPaT (signal phase and timing) feed, a vision-only system cannot know when a stale green turns yellow; Remark 10 applies, and the "stale" aspect of the warning reduces to pure geometry.
2) **Perception uncertainty.** Neural network outputs are point estimates, and a confidence score is not a formal bound. A bounding-box jitter of ± 1.5 m at 14 m/s shifts the arrival-time estimate by roughly 100 ms, which can flip a safe decision into a blocked one. The formal results therefore hold for the given inputs, not for the unobserved ground truth.

TABLE IV

KNOWN VULNERABILITIES AND OPERATIONAL BOUNDARIES.

#	Stage	Vulnerability	Operational bound
1	Decision	Signal timing	V2I / SPaT; otherwise worst-case geometry (Remark 10)
2	Perception	Bounding-box jitter	± 1.5 m $\approx$ 100 ms; a confidence score is not a formal bound
3	Decision	Lead-vehicle motion	leader deceleration $\leq 0.4g$ (Remark 20)
4	HMI	Trust erosion	false positives get ignored; dynamic threshold by reaction time
5	Perception	Occlusion	stop line hidden; outside the formal scope
6	Decision	Road friction	grip fixed at $4.0\ \mathrm{m/s^2}$

3) **Lead-vehicle motion.** Theorem 19 is conditional on a bounded leader deceleration (Remark 20); the operational envelope is defined by that bound.
4) **HMI trust.** Frequent false positives erode driver trust and can cause the system to be ignored, turning the next missed warning into a fatal outcome. The threshold policy is therefore recall-first, minimizing false negatives, and future work targets a dynamic threshold adapted to the driver's reaction time, with warning intensity modulated by confidence.
5) **Occlusion.** A large vehicle can hide the stop line or the lead vehicle entirely; such detection gaps lie outside the formal scope.
6) **Road friction.** Assumption 9 fixes grip at $4.0\ \mathrm{m/s^2}$; wet or icy surfaces reduce it, and the stopping envelope must be re-tuned for the local condition.

c) Scope of the formal results.: The proofs establish properties of the decision logic given exact inputs. They do not bound perception error, signal-timing uncertainty, or actuator performance. Each theorem is a conditional statement: if the inputs satisfy the assumptions, the output is correct. Guaranteeing the inputs themselves (object detection, tracking, signal timing) is a perception problem, and the deterministic engine is deliberately agnostic to how the inputs are obtained. When V2I is unavailable, a vision-based signal-phase classifier [10] can supply the phase and a conservative estimate of time-to-red; any reduction in certainty degrades gracefully to the worst-case bound of Remark 10. Moving the physical bounds into the type system (e.g., constructors that reject impossible speeds or distances) is planned future work and would push a further part of this verification into the compiler.

d) Pedagogical use case.: Because the engine is white box, the HMI can display the violated constraint directly, for example "you are braking at $0.3\,g$, you need $0.5\,g$ to stop before the line". This turns a warning into an explanation, which is valuable for student drivers and differentiates the system from OEM ADAS that treat the driver as a passive monitor.

e) Design boundaries.: Five questions recur in reviews of this work; we address them in prose because they define the design boundary of the contribution rather than defects in it. The perception bottleneck is a scoping choice, not an omission: every theorem is conditional ("if the inputs satisfy the assumptions, the output is correct"), perception error enters through the inputs, and its effect is bounded in Section VII (a ± 1.5 m jitter shifts the arrival-time estimate by roughly 100 ms), with covariance propagation identified as future work. Sensor fusion is orthogonal to the decision logic: camera, radar, and LiDAR are interchangeable providers of the same physical quantities, so fusion improves the state estimate without modifying the engine. Road friction is a configuration parameter (Section VI), not an implicit constant, and the envelope is re-tuned for wet or icy conditions. A tailgating follower is a different control problem: the brake decision is recall-first for the ego vehicle and its leader (Theorem 19), and rear-end protection requires following-distance awareness outside the decision logic. Finally, we agree with the guardian-angel framing: the system is a formally verified safety envelope that vetoes unsafe suggestions, and proving the upstream vision pipeline, through interval or conformal bounds on detector outputs [15], is the natural next step.

f) Comparison with threshold baselines.: A naive rule that warns whenever speed exceeds a constant or distance falls below a constant cannot represent the coupled feasibility region of Figure 2: the feasibility condition is a conjunction, $d_{\mathrm{s}} \leq d_{\mathrm{req}}(v_{\mathrm{e}})$ (stop) or $d_{\mathrm{s}} \geq v_{\mathrm{e}}(t_{\mathrm{y}} - \epsilon) - L_{\mathrm{i}}$ (clear), monotone in both variables. Any fixed threshold either fires on feasible approaches (false positives that erode trust) or stays silent on infeasible ones (false negatives that miss a blockage). The kinematic rule is parameter-free, reproduces the exact boundary, and therefore dominates fixed thresholds on the canonical suite at zero tuning cost.

g) Ethics and safety.: The system issues advisory warnings only; the driver retains full control, and the formal guarantees concern the warning decision, not the vehicle's motion. False positives are a trust hazard (Section VII) and are minimised by construction: the severity-ordered pipeline fires the most severe applicable rule, and the recall-first threshold policy keeps false negatives at zero within the operational envelope. Certification under ISO 26262 [6] would require the deterministic core to run on a safety-rated hardware platform with ASIL-rated integration; the determinism and exhaustiveness of the decision function make such an argument tractable.

h) Limitations.: The framework inherits the limitations of its assumptions. Assumption 7 ignores acceleration and deceleration by surrounding vehicles; Assumption 8 requires signal-timing observability; and Assumption 9 assumes nominal friction. Sensor noise in the perception layer propagates into the physical estimates, and although the decision logic is deterministic, the measurements feeding it are not. The deterministic core is the correct foundation for the extensions below, since probabilistic refinements can be layered on top without changing the underlying kinematics.

i) Future work.: Four extensions follow directly from the limitations above: (i) probabilistic and set-based propagation of perception uncertainty through the kinematics, using Kalman covariances, interval arithmetic, or conformal prediction bounds on detector outputs [15]; (ii) dynamic warning thresholds adapted to the driver's reaction time and to estimated road friction from rain or temperature sensors; (iii) extension of the rule set to vulnerable road users and multi-lane crossing scenes; and (iv) a simulation-based evaluation campaign in SUMO or CARLA [13], [12] with randomised intersection approaches and baseline comparison, complementing the exhaustive verification suite.

VIII. Conclusion

A deterministic, mathematically proven system for intersection blockage prediction has been presented. The method requires no labelled data and provides interpretable warnings based on violated kinematic constraints, formalised in five theorems with complete proofs (Appendix A), valid within an explicitly defined operational envelope (Section VII). The Rust implementation is modular, SOLID-compliant, dependency-free, and severity-ordered, with $O(n)$ per-frame cost and exhaustive handling of sensor states. Source listings, this paper, and the proofs appendix are available online.

Acknowledgment

The author thanks the CivicSense community for field data and feedback. Source code, this paper (PDF and LaTeX), and the HTML proofs appendix are available at https://github.com/arpanpathak/driving-civicsense-vision-model/tree/main/research_paper and rendered at https://arpanpathak.github.io/driving-civicsense-vision-model/.

Appendix A
Complete Mathematical Proofs

This appendix develops the formal results of Section IV self-contained. We first prove supporting lemmas, then restate and prove each theorem in full. Throughout, time is measured in seconds, distances in metres, speeds in $\mathrm{m/s}$, and accelerations in $\mathrm{m/s^2}$. We assume the kinematic model of Assumptions 7–9: longitudinal motion is governed by $\ddot{x} = a(t)$ with $|a(t)| \le a_\mathrm{b}$ and an actuation delay t_r during which $a(t) = 0$.

a) Scope of the proofs.: Every result below is a conditional statement: given exact inputs that satisfy the stated assumptions, the conclusion follows. The proofs do not bound perception error, signal-timing uncertainty, or actuator performance, and they do not assert that the measured inputs equal the ground truth. The formal guarantee therefore covers the decision logic itself, not the perception chain that feeds it.

A. Supporting Lemmas

Lemma 23 (Braking distance)**.** *Under constant deceleration a_b from initial speed v_e, the distance travelled until rest is $v_\mathrm{e}^2/(2a_\mathrm{b})$.*

Proof. The equation of motion during braking is $\ddot{x} = -a_\mathrm{b}$, with initial conditions $x(0) = 0$, $\dot{x}(0) = v_\mathrm{e}$. Integrating twice: $\dot{x}(t) = v_\mathrm{e} - a_\mathrm{b}t$ and $x(t) = v_\mathrm{e}t - \frac{1}{2}a_\mathrm{b}t^2$. The vehicle is at rest when $\dot{x}(t_s) = 0$, i.e. $t_s = v_\mathrm{e}/a_\mathrm{b}$. Substituting:

$$x(t_s) = v_\mathrm{e} \cdot \frac{v_\mathrm{e}}{a_\mathrm{b}} - \frac{1}{2}a_\mathrm{b} \cdot \frac{v_\mathrm{e}^2}{a_\mathrm{b}^2} = \frac{v_\mathrm{e}^2}{a_\mathrm{b}} - \frac{v_\mathrm{e}^2}{2a_\mathrm{b}} = \frac{v_\mathrm{e}^2}{2a_\mathrm{b}}.$$

□

Lemma 24 (Reaction distance)**.** *During the reaction delay t_r, during which no braking is applied, the vehicle travels $v_\mathrm{e}t_\mathrm{r}$ at constant speed.*

Proof. With $a(t) = 0$ for $t \in [0, t_\mathrm{r}]$, $\dot{x} = v_\mathrm{e}$ is constant, hence $x(t_\mathrm{r}) = v_\mathrm{e}t_\mathrm{r}$. □

Lemma 25 (Intermediate value for braking profiles)**.** *Let $x_\alpha(t)$ be the trajectory obtained by applying deceleration $\alpha \in [0, a_\mathrm{b}]$ after the reaction delay. Then the stopping position $x_\alpha(\infty) = v_\mathrm{e}t_\mathrm{r} + v_\mathrm{e}^2/(2\alpha)$ (with the convention that $\alpha = 0$ means "never stops") is continuous and strictly decreasing in α on $(0, a_\mathrm{b}]$.*

Proof. By Lemma 23, the braking contribution is $v_\mathrm{e}^2/(2\alpha)$, which is continuous and strictly decreasing in α on $(0, a_\mathrm{b}]$; the reaction contribution $v_\mathrm{e}t_\mathrm{r}$ is independent of α. Hence the total stopping position is continuous and strictly decreasing in α. □

Corollary 26 (Exact-stop profile)**.** *If $d_\mathrm{s} > d_\mathrm{req}(v_\mathrm{e})$, there exists a deceleration $\alpha^* \in (0, a_\mathrm{b})$ such that the vehicle stops exactly at the stop line: $x_{\alpha^*}(\infty) = d_\mathrm{s}$.*

Proof. By Lemma 25, the map $\alpha \mapsto x_\alpha(\infty)$ is continuous on $(0, a_\mathrm{b}]$, with $x_{a_\mathrm{b}}(\infty) = d_\mathrm{req}(v_\mathrm{e}) < d_\mathrm{s}$ and $\lim_{\alpha \to 0^+} x_\alpha(\infty) = +\infty > d_\mathrm{s}$. The intermediate value theorem yields $\alpha^* \in (0, a_\mathrm{b})$ with $x_{\alpha^*}(\infty) = d_\mathrm{s}$. □

Lemma 27 (Minimum-time position bound)**.** *Any trajectory with $|\ddot{x}| \le a_\mathrm{b}$ that begins at $x(0) = 0$ with $\dot{x}(0) = v_\mathrm{e}$ and applies no braking before t_r satisfies $x(t) \ge x_\mathrm{min}(t)$ for all t, where x_min is the maximum-braking trajectory.*

Proof. For $t \in [0, t_\mathrm{r}]$ all admissible trajectories coincide ($x = v_\mathrm{e}t$). For $t > t_\mathrm{r}$, the trajectory with the largest deceleration $a = -a_\mathrm{b}$ has the smallest velocity and hence the smallest position at every subsequent time; any smaller deceleration (or acceleration) yields $x(t) \ge x_\mathrm{min}(t)$ by monotone comparison of the integrated velocity. □

B. Proof of Theorem 11 (Stopping distance)

Proof. The admissible trajectory is the concatenation of the reaction phase (no braking, $t \in [0, t_\mathrm{r}]$) and the braking phase (maximal deceleration a_b). By Lemma 24 the reaction phase contributes $v_\mathrm{e}t_\mathrm{r}$; by Lemma 23 the braking phase contributes $v_\mathrm{e}^2/(2a_\mathrm{b})$. No braking profile can do better: by Lemma 27, any trajectory that stops must cover at least the distance covered by the maximum-braking trajectory, and delaying braking beyond t_{r} or braking at less than a_{b} only increases the stopping distance (Lemma 25). Hence $d_{\mathrm{req}}(v_{\mathrm{e}}) = v_{\mathrm{e}} t_{\mathrm{r}} + v_{\mathrm{e}}^2/(2a_{\mathrm{b}})$ is the exact minimum. □

C. Proof of Theorem 12 (Stopping feasibility)

Proof. ($\Rightarrow$) Suppose a safe stop exists: some trajectory x with $|a| \leq a_{\mathrm{b}}$ and $a(t) = 0$ for $t \in [0, t_{\mathrm{r}}]$ satisfies $x(t) < d_{\mathrm{s}}$ for all t and $\dot{x}(t) \to 0$. By Lemma 27, $x(t) \geq x_{\min}(t)$, hence $d_{\mathrm{s}} > \sup_t x(t) \geq x_{\min}(\infty) = d_{\mathrm{req}}(v_{\mathrm{e}})$. Thus $d_{\mathrm{s}} > d_{\mathrm{req}}(v_{\mathrm{e}})$.

($\Leftarrow$) Suppose $d_{\mathrm{s}} > d_{\mathrm{req}}(v_{\mathrm{e}})$. By Corollary 26 there exists $\alpha^* \in (0, a_{\mathrm{b}})$ such that the trajectory stops exactly at d_{s}. This trajectory satisfies $x(t) \leq d_{\mathrm{s}}$ with equality only at the stopping instant, so it is a safe stop. Hence stopping feasibility is equivalent to $d_{\mathrm{s}} > d_{\mathrm{req}}(v_{\mathrm{e}})$. □

D. Proof of Theorem 14 (Clearance feasibility)

Proof. Under the constant-velocity policy of Assumption 7, the longitudinal position is $x(t) = v_{\mathrm{e}} t$ for $t \geq 0$. The far boundary $d_{\mathrm{s}} + L_{\mathrm{i}}$ is reached exactly at $t = t_{\mathrm{c}} = (d_{\mathrm{s}} + L_{\mathrm{i}})/v_{\mathrm{e}}$.

($\Rightarrow$) If the vehicle clears before the red phase, then $x(t_{\mathrm{y}} - \epsilon) \geq d_{\mathrm{s}} + L_{\mathrm{i}}$ (Definition 3 requires occupancy only for $t \geq t_{\mathrm{y}} - \epsilon$; clearing means no occupancy thereafter). Since x is strictly increasing, $t_{\mathrm{c}} \leq t_{\mathrm{y}} - \epsilon$. Requiring strict inequality $t_{\mathrm{c}} < t_{\mathrm{y}} - \epsilon$ guarantees that the entire vehicle has exited with margin to spare.

($\Leftarrow$) If $t_{\mathrm{c}} < t_{\mathrm{y}} - \epsilon$, then $x(t_{\mathrm{y}} - \epsilon) = v_{\mathrm{e}}(t_{\mathrm{y}} - \epsilon) > v_{\mathrm{e}} \cdot t_{\mathrm{c}} = d_{\mathrm{s}} + L_{\mathrm{i}}$, so the vehicle has fully crossed the far boundary strictly before the margin instant and hence before the red phase. □

E. Proof of Theorem 16 (Dilemma zone)

Proof. We prove the two implications separately, using the equivalence of stopping feasibility (Theorem 12) and clearance feasibility (Theorem 14) under Assumption 7.

Sufficiency. Assume $d_{\mathrm{s}} \leq d_{\mathrm{req}}(v_{\mathrm{e}})$ and $t_{\mathrm{c}} \geq t_{\mathrm{y}} - \epsilon$. By Theorem 12, no safe stop exists; by Theorem 14, no safe clearance exists. Consider any admissible trajectory. If it never crosses the stop line, it must eventually stop (Assumption 7 leaves only braking as a way to remain bounded); but stopping is impossible without crossing the line by the first conjunct, contradiction. Therefore every trajectory crosses the stop line, and since it cannot clear before $t_{\mathrm{y}} - \epsilon$ (second conjunct), it occupies the box $[d_{\mathrm{s}}, d_{\mathrm{s}} + L_{\mathrm{i}}]$ at some $t \geq t_{\mathrm{y}} - \epsilon$. By Definition 3 the vehicle is blocked under every admissible trajectory, so a CRITICAL warning is necessary.

Necessity. Suppose a CRITICAL warning is required, i.e., blockage is unavoidable for every admissible trajectory. If a safe stop were possible ($d_{\mathrm{s}} > d_{\mathrm{req}}(v_{\mathrm{e}})$), then by Theorem 12 the driver could avoid blockage by stopping; contradiction. Hence $d_{\mathrm{s}} \leq d_{\mathrm{req}}(v_{\mathrm{e}})$. Similarly, if a safe clearance were possible ($t_{\mathrm{c}} < t_{\mathrm{y}} - \epsilon$), the driver could avoid blockage by clearing; contradiction. Hence $t_{\mathrm{c}} \geq t_{\mathrm{y}} - \epsilon$. Both conjuncts of (3) hold, so the criterion is also sufficient for the warning. □

Proof of Corollary 17. With the constants of Table I and $t_{\mathrm{y}} = 3.5$ s, the clearance boundary is $d_{\mathrm{s}} = 2.7v_{\mathrm{e}} - 16$ and the stopping boundary is $d_{\mathrm{req}}(v_{\mathrm{e}}) = v_{\mathrm{e}} + v_{\mathrm{e}}^2/8$. Their difference is $d_{\mathrm{req}}(v_{\mathrm{e}}) - (2.7v_{\mathrm{e}} - 16) = v_{\mathrm{e}}^2/8 - 1.7v_{\mathrm{e}} + 16$, a convex quadratic with discriminant $(-1.7)^2 - 4(1/8)(16) = 2.89 - 8 = -5.11 < 0$ and positive leading coefficient. Hence $d_{\mathrm{req}}(v_{\mathrm{e}}) > 2.7v_{\mathrm{e}} - 16$ for all v_{e}, so the dilemma band $\{(v_{\mathrm{e}}, d_{\mathrm{s}}) : 2.7v_{\mathrm{e}} - 16 \leq d_{\mathrm{s}} \leq d_{\mathrm{req}}(v_{\mathrm{e}})\}$ is non-empty for every $v_{\mathrm{e}} \geq 5.93$ (below which the lower bound is negative and the band extends to $d_{\mathrm{s}} = 0$). □

F. Proof of Theorem 19 (Following blockage)

Proof. Let $x_{\mathrm{lead}}(t)$ denote the leader's longitudinal position. The collision-avoidance constraint is $x_{\mathrm{ego}}(t) \leq x_{\mathrm{lead}}(t) - \Delta$ for some positive stand-off Δ; the analysis is insensitive to $\Delta \geq 0$, so take $\Delta = 0$ for the bound. The leader's clearance time is $t_{\mathrm{lead}} = (d_{\mathrm{l}} + L_{\mathrm{i}})/v_{\mathrm{l}}$, which is exactly $t_{\mathrm{c}}^{\mathrm{eff}}$ of Definition 18 (the small constant δ prevents division by zero when $v_{\mathrm{l}} = 0$ and makes the inequality strict in the border case). If $t_{\mathrm{c}}^{\mathrm{eff}} \geq t_{\mathrm{y}} - \epsilon$, the leader occupies the box at $t = t_{\mathrm{y}} - \epsilon$, i.e., $x_{\mathrm{lead}}(t_{\mathrm{y}} - \epsilon) < d_{\mathrm{l}} + L_{\mathrm{i}}$. By the collision-avoidance constraint, $x_{\mathrm{ego}}(t_{\mathrm{y}} - \epsilon) \leq x_{\mathrm{lead}}(t_{\mathrm{y}} - \epsilon) < d_{\mathrm{l}} + L_{\mathrm{i}} \leq d_{\mathrm{s}} + L_{\mathrm{i}}$ (using $d_{\mathrm{l}} \leq d_{\mathrm{s}}$), so the ego has not cleared the far boundary by the margin instant; moreover the ego has already crossed the stop line, since it cannot stop (see below). Hence the ego is blocked (Definition 3).

It remains to show the ego cannot abort. At the decision instant the gap to the leader is $d_{\mathrm{l}} < d_{\mathrm{req}}(v_{\mathrm{e}})$. By Theorem 12, any braking trajectory that halts the ego does so at a position at least $d_{\mathrm{req}}(v_{\mathrm{e}})$ beyond the decision point, i.e., at or beyond the leader's current position; since the leader is moving forward (or is stopped), the ego cannot come to rest strictly behind the leader without violating the collision-avoidance constraint. The ego therefore cannot stop in time and must proceed, which we have shown leads to blockage. □

G. Proof of Theorem 22 (Cut-in warning)

Proof. By Definition 21, the intruding vehicle crosses the lane boundary at $t = t_{\mathrm{i}} = W_{\mathrm{l}}/|v_{\mathrm{lat}}|$. If $t_{\mathrm{i}} < t_{\mathrm{y}}$, the intrusion occurs strictly before the signal transition. At the intrusion instant, the longitudinal separation between ego and intruder is at most d_{a} (the intruder is ahead at the ego's lane entry point; if it enters behind the ego the situation is not safety-relevant, so consider the ahead case). Since $d_{\mathrm{a}} < d_{\mathrm{req}}(v_{\mathrm{e}})$, by Theorem 12 the ego cannot brake to a halt before reaching the intruder's position. Two cases arise:

Case 1 (ego brakes): the ego cannot stop before the intruder, so it either collides with the intruder or, if the intruder is beyond the stop line, stops beyond the stop line and occupies the box. Both outcomes violate Definitions 3 or safe operation.

Case 2 (ego proceeds): the intruder now occupies the ego's lane and acts as a slow lead vehicle; by Theorem 19 with the intruder as leader (its gap $d_{\mathrm{a}} < d_{\mathrm{req}}(v_{\mathrm{e}})$ and its speed implying a clearance failure), blockage follows.

In either case the kinematic constraints are violated and a warning is required. □

H. Soundness of the Decision Pipeline

Lemma 28 (Pipeline soundness)**.** *Let* ℓ^* *be the output of* `evaluate_safety` *on any scene. Then* $\ell^* = \max_{\mathit{severity}}\{\ell_r\}$ *over all rules* r *that fire, where the severity order is* SAFE $<$ CAUTION $<$ WARNING $<$ CRITICAL*; if no rule fires,* $\ell^* =$ SAFE*.*

Proof. The pipeline evaluates the rules in the fixed order [`rule_light`, `rule_dilemma`, `rule_lead`, `rule_cutin`, `rule_stale`], and `find_map` returns the first element that evaluates to `Some`. Inspection of each rule shows its maximum severity level: `rule_light` and `rule_dilemma` can emit CRITICAL (the highest); `rule_lead` can emit CRITICAL (subrule 3a) or WARNING; `rule_cutin` emits at most WARNING; `rule_stale` emits at most CAUTION. Because the pipeline order is non-increasing in the severity bound of each rule, the first firing rule carries the maximum severity among all firing rules. If none fires, `unwrap_or(Safe)` returns SAFE. □

Corollary 29 (Red-light dominance)**.** *If* `LightState::Red`*, the output is* CRITICAL *regardless of all other inputs.*

Proof. `rule_light` maps `Red` to CRITICAL and is evaluated first; by Lemma 28 the output is the maximum severity, i.e., CRITICAL. □

REFERENCES

[1] D. Gazis, R. Herman, and A. Maradudin, "The problem of the amber signal light in traffic flow," *Oper. Res.*, vol. 8, no. 1, pp. 112–132, 1960.
[2] D. A. Redelmeier and R. J. Tibshirani, "Association between cellular-telephone calls and motor vehicle collisions," *N. Engl. J. Med.*, vol. 336, no. 7, pp. 453–458, 1997.
[3] S. Shalev-Shwartz, S. Shammah, and A. Shashua, "On a formal model of safe and scalable self-driving cars," *arXiv preprint arXiv:1708.06374*, 2017.
[4] J. Philion and S. Fidler, "Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3D," in *Proc. ECCV*, 2020, pp. 194–210.
[5] N. Matsakis and F. Klock, "The Rust language," *ACM SIGAda Ada Letters*, vol. 34, no. 3, pp. 103–104, 2014.
[6] ISO, *Road vehicles – Functional safety*, ISO 26262:2018, 2018.
[7] C. V. Zegeer and R. C. Deen, "Green-extension systems at high-speed intersections," *ITE J.*, vol. 48, no. 11, pp. 19–24, 1978.
[8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, 2016, pp. 779–788.
[9] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *Proc. IEEE ICIP*, 2017, pp. 3645–3649.
[10] K. Behrendt and L. Novak, "A deep learning approach to traffic lights: Detecting, tracking, and classifying traffic lights during nighttime," in *Proc. IEEE IV*, 2017, pp. 1244–1249.
[11] M. Althoff and J. M. Dolan, "Online verification of automated road vehicles using reachability analysis," *IEEE Trans. Robot.*, vol. 30, no. 4, pp. 903–918, 2014.
[12] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. Conf. on Robot Learning*, 2017, pp. 1–16.
[13] P. A. Lopez et al., "Microscopic traffic simulation using SUMO," in *Proc. IEEE ITSC*, 2018, pp. 2575–2582.
[14] C. Godard, O. Mac Aodha, M. Firman, and G. J. Brostow, "Digging into self-supervised monocular depth estimation," in *Proc. IEEE ICCV*, 2019, pp. 3828–3838.
[15] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.
[16] S. Klabnik and C. Nichols, *The Rust Programming Language*, 2nd ed. San Francisco, CA, USA: No Starch Press, 2019.